# 大作业实验报告

无 48 吴泓锦 2024010654

## 1.1

1. LED 2. PWM 波控制 3. OLED 4. 3.3 5. I2C 协议通讯

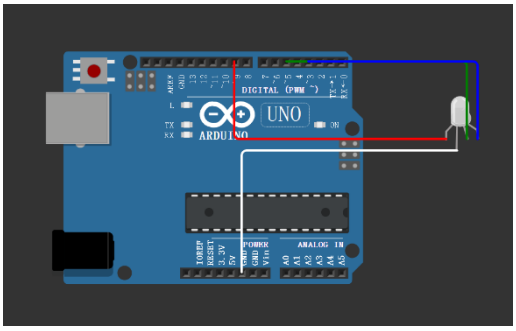
6. 5 7. 舵机 8. PWM 波控制 9. 蜂鸣器 10. PWM 波控制

11. 模拟 12. GND 13. 5 14. 距离 15. 上拉 16. 数字（输入输出）

17. L298N 18. PWM 波控制 ENA 等，数字信号直接控制 IN1 等 19. 编码 20. 12

## 1.2

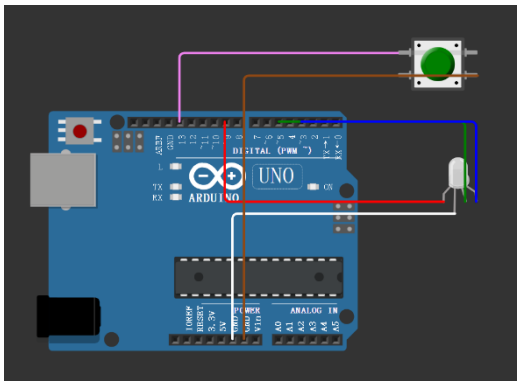
**【Ex 1.2.1】**




**代码思路：**

最简单的呼吸灯功能，将其中一颜色对应串口进行 PWM 输出，占空比在 0~255~0 滚动一遍（本程序用了较简单的 delay），另两颜色对应串口输出 0 电平，并轮流循环对三色进行上述操作即可。

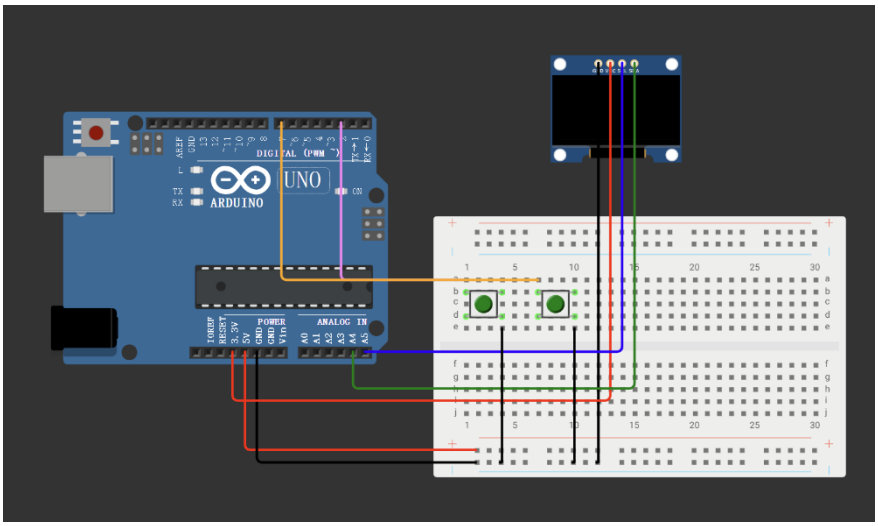
**【Ex 1.2.2】**




**代码思路：**

与作业 3.2 基本类似，不同点在于引入 sequence 变量切换颜色，每次亮度重新减小到 0 之后，sequence++，利用其除 3 余数判断给蓝绿红哪个引脚输出 PWM，其余引脚为 0 电平。

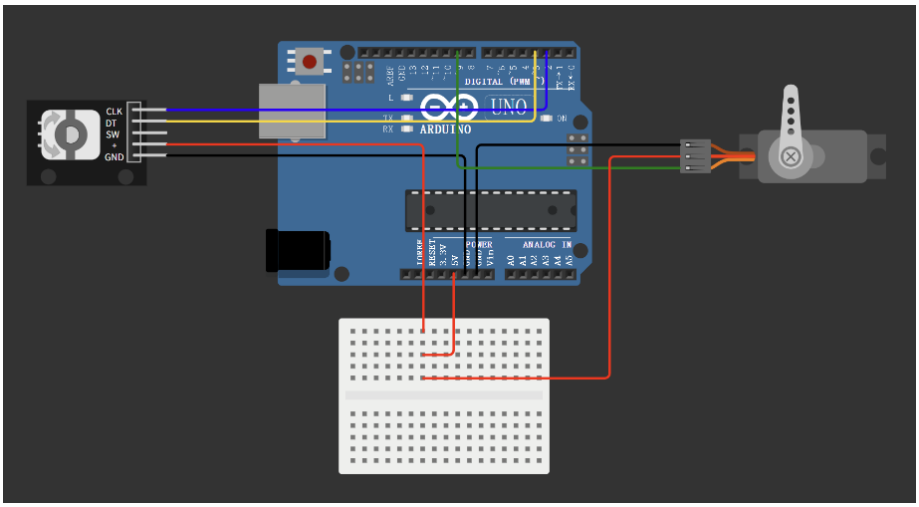
【Ex 1.2.3】



**代码思路：**

在 setup 中给 OLED 输出原有 target 值 100，在 loop 中检测按钮状态，并将两个按钮的单击分别绑定到 upclick 和 downclick 两个函数，两个函数的内容均为对 target 值进行加减并在 OLED 上显示出该值。

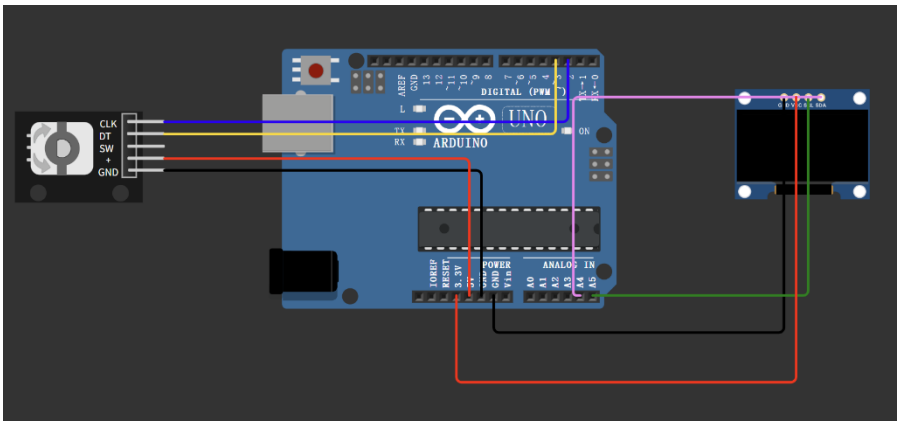
【Ex 1.2.4】




**代码思路：**

将编码器两相的电平变化分别绑定至一个中断函数，中断函数检测两相电平是否相同，据此进行四倍频计数。通过减速比等参数计算出电机旋转角度，并对 360 取余（保证角度在 0 到 360 之间），将该角度线性映射到 0 到 180 之间，对舵机输出该角度即可。

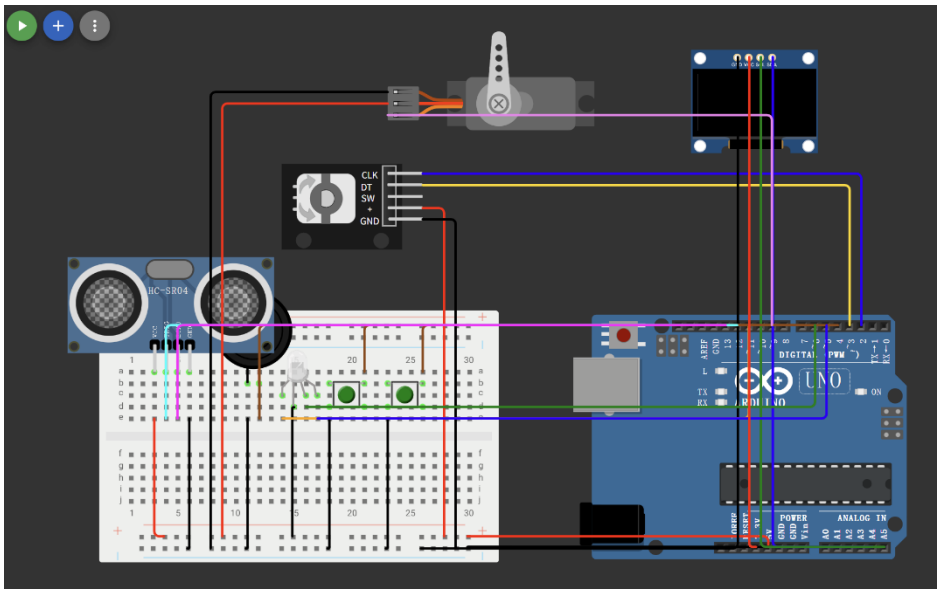
【Ex 1.2.5】




**代码思路：**

与 1.2.4 类似计算出电机旋转角度，并线性映射到 0 至 100（作为进度条填充百分比 progress），画一个内宽 100 的矩形框（drawFrame），根据 progress 进行填充（drawBox）。

**【Ex 2、3】（Ex 2 实现所有功能，Ex 3 实现 3.3~3.6 共四项）**




**代码思路：**

**整体思路：**合理安排引脚的分配（特别注意舵机对定时器的占用，9 和 10 最多其一连舵机），两个重要的全局变量 currentstate 和 functionstate 分别指示现在的模式（0 至 6 依次代表开始、菜单、测距、测电压、示波器、音乐、手电筒）和模式内的功能（具体可见代码内的注释，此处不再赘述）。在两个按钮的单击、双击、长按所绑定的函数中进行 currentstate、functionstate 和其他全局变量的修改，在 loop 函数内通过 tick()检测两个按钮状态，并根据 currentstate 调用对应的模式函数（七个模式分别对应一个函数），在相应所调用的模式函数中根据 functionstate 执行相应功能。

**开始（Ex 2.1 3.5）**：设计了一个 HELLO 从屏幕左侧滑入、直至屏幕右侧滑出并循环的动画，采用 animState 全局变量表示状态（0 开始、1 滑入中、2 滑出中、3 完全滑出）。利用 millis()判断 HELLO 在每个横坐标处停留时长是否足够，若足够，根据 animState 选择对横坐标增加 5 或复原到起始位，并在新的坐标处显示 HELLO。

**菜单（Ex 2.2）**：如果 currentstate=1，则在 loop 中根据电机角度计算所选择的项目编号 select（全局变量）。在菜单函数中，显示 MENU 和五个功能的标题，并根据 select 在相应功能标题右侧画一个向左的小箭头。（在按钮 1 双击绑定的函数中，如果 currentstate=1，根据 select 更改 currentstate）。

**测距（Ex 2.5 2.6）**：根据超声波模块原理获得距离并显示在 OLED 上（根据距离大小选择保留几位小数，并利用 dtoftrf 转化为字符串，使得整体保留两位有效数字）。

functionstate=1 时，计算电机旋转角度更改阈值，并进行基础呼吸灯操作（利用 millis()）。functionstate=0 时，如果距离大于阈值，对绿灯引脚输出高电平，如果距离小于阈值，利用全局变量 ifwarn，为真时红灯引脚输出高电平，蜂鸣器输出声音，为假时不输出，且当 millis()与上次记录的时间戳相差超过 interval 时对 ifwarn 取反，距离越小 interval 越小。

**测电压（Ex 2.4 3.6）**：从 A0 串口利用 analogRead 读入，线性映射至 0~5 得到电压值，通过 dtostrf 转化字符串并在 OLED 上显示。将读入值线性映射至 10~170 得到舵机角度并输出（录视频时刻度标反了）。在合适位置利用 drawCircle 画一个圆作电压表表盘，并利用循环画上刻度，以圆心为其中一端点画线段作指针，另一端点由电压值线性映射并经三角函数运算得到。

**示波器（Ex 2.7 2.8 2.9）**：先利用 drawLine 画两坐标轴和箭头，根据 functionstate 选择输入串口。利用全局变量 currentx 表示当前画到的横坐标，coordinates 数组存放读入电压映射得到的纵坐标值。每次调用示波器函数，读入电压得到新的纵坐标，更新 coordinates[currentx]并将 currentx 后移，并对 currentx 之前的部分画出图像（循环画出(i-1, coordinates[i-11])到(i, coordinates[i-10]的线段，因为横坐标 0 在 OLED 横向 10 像素处）。当 currentx=127 时，后移 currentx 时使其为 10，从而清屏重画。

**音乐（Ex 2.10 3.3 3.4）**：利用局部变量储存三首歌的音符和长度（做简化：每个音符时长相同），根据 functionstate 将某一首的数据赋值给 melody 和 len 两个变量。设置 playstate 变量，0-停止，1-正在播某一音符，2-音符播完准备播下一个音符，3-暂停。若 playstate=1，利用 millis()和音符开始时的时间戳之差判断是否播完，如播完令 playstate=2，currentnote++，同时判断节奏闪灯持续时间是否足够，若足够则熄灭（闪灯持续时间小于

单个音符播放时间）。下面判断若 playstate=2，判断 currentnote 和 len 关系，若播完则 playstate=0，若未播完则播发下一音符并对绿灯输出高电平。若 playstate=3，保留 currentnote 等信息，暂停播放，再次单击按钮 2 将 playstate 更改为 2.利用 millis()和 currentnote 计算出播放进度。如果 playstate 不为 0，在 OLED 上显示歌名，绘制进度条。**手电筒（Ex 2.3）**：functionstate 对应 0-熄灭，1-蓝微亮，2-白微亮，3-白最亮，在手电筒函数中根据 functionstate 进行 analogWrite，在按钮 1 长按、按钮 2 单击、按钮 2 双击函数中进行 functionstate 的修改。

**【对课程的建议】**

课程学下来很有收获，尤其是作业以制作小项目或大项目的形式进行，能在完成项目的过程中收获很多（尤其是大作业）。但是，希望助教老师们在课上讲解时多从应用层面讲各种器件怎么用、有哪些可以控制它们的函数（类似于程设的讲法），而少一些从原理层面抽象的剖析（例如 OLED 的使用涉及到很庞杂的知识，课程中几乎未涉及，但作业中经常需要用到且要求很高）。老师、助教们辛苦了！